

PART A
Experiment No.07

A.1 Aim: Implement Monkey Banana Problem using Python

A.2 Prerequisite: Discrete structure

A.3 Outcome:

After successful completion of this experiment students will be able to
. Solve real life problems using AI

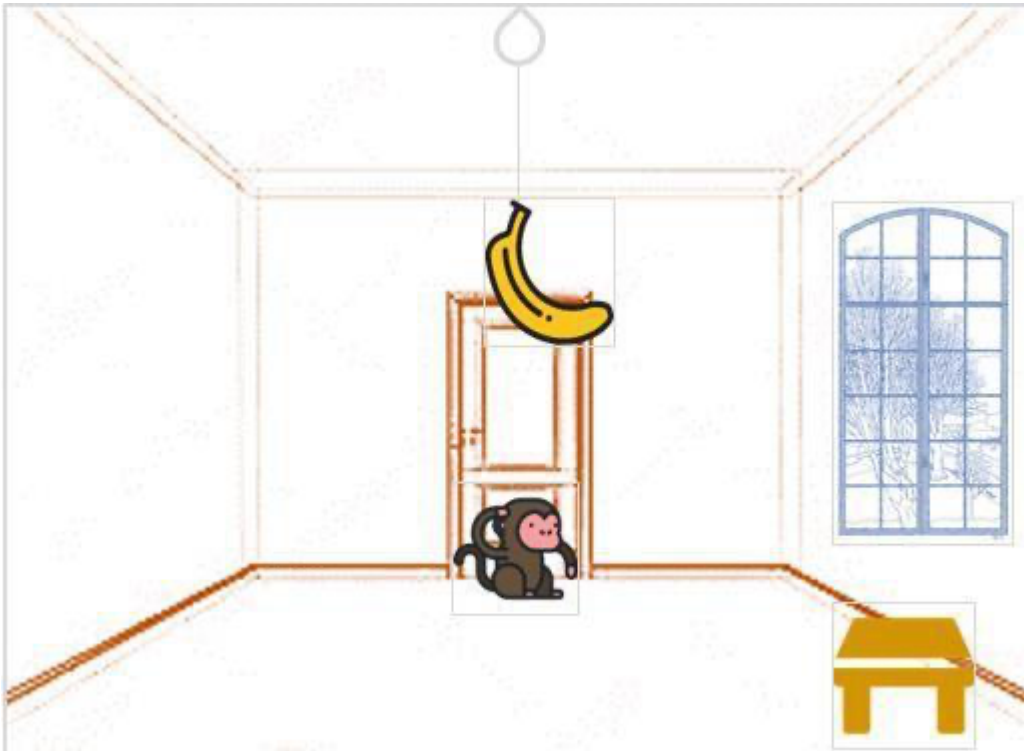
Tools Required: Python /Java

A.4 Theory:

Problem Statement

Suppose the problem is as given below –

- A hungry monkey is in a room, and he is near the door.
- The monkey is on the floor.
- Bananas have been hung from the center of the ceiling of the room.
- There is a block (or chair) present in the room near the window.
- The monkey wants the banana, but cannot reach it.



So how can the monkey get the bananas?

So if the monkey is clever enough, he can come to the block, drag the block to the center, climb on it, and get the banana. Below are few observations in this case –

- Monkey can reach the block, if both of them are at the same level. From the above image, we can see that both the monkey and the block are on the floor.
- If the block position is not at the center, then monkey can drag it to the center.
- If monkey and the block both are on the floor, and block is at the center, then the monkey can climb on the block. So the vertical position of the monkey will be changed.
- When the monkey is on the block, and block is at the center, then the monkey can get the bananas.

Now, let us see how we can solve this using Prolog. We will create some predicates as follows –

We have some predicates that will move from one state to another state, by performing action.

- When the block is at the middle, and monkey is on top of the block, and monkey does not have the banana (i.e. *has not* state), then using the *grasp* action, it will change from *has not* state to *have* state.

- From the floor, it can move to the top of the block (i.e. **on top** state), by performing the action **climb**.
- The **push** or **drag** operation moves the block from one place to another.
- Monkey can move from one place to another using **walk** or **move** clauses.

Another predicate will be `canget()`. Here we pass a state, so this will perform move predicate from one state to another using different actions, then perform `canget()` on state 2. When we have reached to the state '**has>**', this indicates '**has banana**'. We will stop the execution.

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.C-54	Name:Quazi Md Moinuddin
Class: TE Comps C	Batch: C-3
Date of Experiment:15.03.25	Date of Submission:5.04.25
Grade:	

B.1 Software Code written by student:

```
from collections import deque

class MonkeyBananaProblem:
    def __init__(self):
        self.monkey_position = "ground" # The monkey starts on the
ground
        self.banana_position = "hanging" # The banana is hanging
        self.box_position = "ground" # The box is on the ground

    def apply_action(self, state, action):
        """Returns new state after performing the action (moving or
using the box)"""
        monkey_pos, box_pos, banana_pos = state

        if action == "move_box_under_banana" and box_pos ==
"ground":
```

```

        box_pos = "under_banana" # Move the box under the
banana
        elif action == "monkey_climb_box" and box_pos ==
"under_banana" and monkey_pos == "ground":
            monkey_pos = "on_box" # Monkey climbs onto the box
        elif action == "reach_banana" and monkey_pos == "on_box":
            banana_pos = "picked" # Monkey reaches the banana
            return "Goal" # Goal state reached

    return (monkey_pos, box_pos, banana_pos)

def bfs(self):
    """Breadth-First Search to solve the problem"""
    initial_state = ("ground", "ground", "hanging") # Initial
state
    queue = deque([initial_state])
    visited = set([initial_state])

    # Print the initial state
    print(f"Initial State: {initial_state}")

    while queue:
        state = queue.popleft()

        # If the state is the goal
        if state == "Goal":
            print("Goal reached! Monkey got the bananas.")
            return True

        monkey_pos, box_pos, banana_pos = state # Unpack the
state

        # Explore possible actions
        for action in ["move_box_under_banana",
"monkey_climb_box", "reach_banana"]:
            new_state = self.apply_action(state, action)

```

```

        # If the state changes and hasn't been visited yet,
        add it to the queue
        if new_state != state and new_state not in visited:
            visited.add(new_state)
            queue.append(new_state)

        # Output the action and the resulting new state
        print(f"Move: {action} -> State: {new_state}")

    print("No solution found!")
    return False

# Run the BFS to solve the Monkey and Banana problem
problem = MonkeyBananaProblem()
problem.bfs()

```

B.2 Input and Output:

```

➡ Initial State: ('ground', 'ground', 'hanging')
Move: move_box_under_banana -> State: ('ground', 'under_banana', 'hanging')
Move: monkey_climb_box -> State: ('on_box', 'under_banana', 'hanging')
Move: reach_banana -> State: Goal
Goal reached! Monkey got the bananas.
True

```

B.3 Observations and learning:

Initial Setup: The monkey starts on the ground, the box is on the ground, and the banana is hanging.

Actions:

- **Move Box:** Move the box under the banana.
- **Climb Box:** Monkey climbs the box to reach the banana.
- **Reach Banana:** Monkey grabs the banana.

Goal: The goal is achieved when the monkey reaches the banana.

State Transitions:

- Initial state: ('ground', 'ground', 'hanging')
- After moving the box: ('ground', 'under_banana', 'hanging')
- After climbing the box: ('on_box', 'under_banana', 'hanging')
- Goal state: ('on_box', 'under_banana', 'picked')

Efficiency: The BFS explores the simplest solution path, ensuring no redundant actions.

B.4 Conclusion:

The **Monkey and Banana Problem** demonstrates a simple sequence of actions where the monkey uses a box to reach a hanging banana. Using **BFS**, the problem is solved by systematically moving the box, climbing it, and then reaching the banana. The solution is efficient, clear, and demonstrates basic state transitions in problem-solving.